

CHAPTER 12



Physical Storage Systems

In preceding chapters, we have emphasized the higher-level models of a database. For example, at the *conceptual* or *logical* level, we viewed the database, in the relational model, as a collection of tables. Indeed, the logical model of the database is the correct level for database *users* to focus on. This is because the goal of a database system is to simplify and facilitate access to data; users of the system should not be burdened unnecessarily with the physical details of the implementation of the system.

In this chapter, however, as well as in Chapter 13, Chapter 14, Chapter 15, and Chapter 16, we probe below the higher levels as we describe various methods for implementing the data models and languages presented in preceding chapters. We start with characteristics of the underlying storage media, with a particular focus on magnetic disks and flash-based solid-state disks, and then discuss how to create highly reliable storage structures by using multiple storage devices.

12.1 Overview of Physical Storage Media

Several types of data storage exist in most computer systems. These storage media are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability. Among the media typically available are these:

- **Cache.** The cache is the fastest and most costly form of storage. Cache memory is relatively small; its use is managed by the computer system hardware. We shall not be concerned about managing cache storage in the database system. It is, however, worth noting that database implementors do pay attention to cache effects when designing query processing data structures and algorithms, and we shall return to this issue in later chapters.
- **Main memory.** The storage medium used for data that are available to be operated on is main memory. The general-purpose machine instructions operate on main memory. Main memory may contain tens of gigabytes of data on a personal com-

puter, and even hundreds to thousands of gigabytes of data in large server systems. It is generally too small (or too expensive) for storing the entire database for very large databases, but many enterprise databases can fit in main memory. However, the contents of main memory are lost in the event of a power failure or system crash; main memory is therefore said to be **volatile**.

- **Flash memory.** Flash memory differs from main memory in that stored data are retained even if power is turned off (or fails)—that is, it is **non-volatile**. Flash memory has a lower cost per byte than main memory, but a higher cost per byte than magnetic disks.

Flash memory is widely used for data storage in devices such as cameras and cell phones. Flash memory is also used for storing data in “USB flash drives,” also known as “pen drives,” which can be plugged into the *Universal Serial Bus* (USB) slots of computing devices.

Flash memory is also increasingly used as a replacement for magnetic disks in personal computers as well as in servers. A **solid-state drive (SSD)** uses flash memory internally to store data but provides an interface similar to a magnetic disk, allowing data to be stored or retrieved in units of a block; such an interface is called a *block-oriented interface*. Block sizes typically range from 512 bytes to 8-kilobytes. As of 2018, a 1-terabyte SSD costs around \$250. We provide more details about flash memory in Section 12.4.

- **Magnetic-disk storage.** The primary medium for the long-term online storage of data is the magnetic disk drive, which is also referred to as the **hard disk drive (HDD)**. Magnetic disk, like flash memory, is non-volatile: that is, magnetic disk storage survives power failures and system crashes. Disks may sometimes fail and destroy data, but such failures are quite rare compared to system crashes or power failures.

To access data stored on magnetic disk, the system must first move the data from disk to main memory, from where they can be accessed. After the system has performed the designated operations, the data that have been modified must be written to disk.

Disk capacities have grown steadily over the years. As of 2018, the size of magnetic disks ranges from 500 gigabytes to 14 terabytes, and a 1-terabyte disk costs about \$50, while an 8-terabyte disk around \$150. Although significantly cheaper than SSDs, magnetic disks provide lower performance in terms of number of data access operations that they can support per second. We provide further details about magnetic disks in Section 12.3.

- **Optical storage.** The *digital video disk* (DVD) is an optical storage medium, with data written and read back using a laser light source. The *Blu-ray DVD* format has a capacity of 27 gigabytes to 128 gigabytes, depending on the number of layers supported. Although the original (and still main) use of DVDs was to store video data, they are capable of storing any type of digital data, including backups of

database contents. DVDs are not suitable for storing active database data since the time required to access a given piece of data can be quite long compared to the time taken by a magnetic disk.

Some DVD versions are read-only, written at the factory where they are produced, other versions support *write-once*, allowing them to be written once, but not overwritten, and some versions can be rewritten multiple times. Disks that can be written only once are called **write-once, read-many** (WORM) disks.

Optical disk **jukebox** systems contain a few drives and numerous disks that can be loaded into one of the drives automatically (by a robot arm) on demand.

- **Tape storage.** Tape storage is used primarily for backup and archival data. *Archival* data refers to data that must be stored safely for a long period of time, often for legal reasons. Magnetic tape is cheaper than disks and can safely store data for many years. However, access to data is much slower because the tape must be accessed sequentially from the beginning of the tape; tapes can be very long, requiring tens to hundreds of seconds to access data. For this reason, tape storage is referred to as **sequential-access** storage. In contrast, magnetic disk and SSD storage are referred to as **direct-access** storage because it is possible to read data from any location on disk.

Tapes have a high capacity (1 to 12 terabyte capacities are currently available), and can be removed from the tape drive. Tape drives tend to be expensive, but individual tapes are usually significantly cheaper than magnetic disks of the same capacity. As a result, tapes are well suited to cheap archival storage and to transferring large amounts of data between different locations. Archival storage of large video files, as well as storage of large volumes of scientific data, which can range up to many petabytes (1 petabyte = 10^{15} bytes) of data, are two common use cases for tapes.

Tape libraries (jukeboxes) are used to hold large collections of tapes, allowing automated storage and retrieval of tapes without human intervention.

The various storage media can be organized in a hierarchy (Figure 12.1) according to their speed and their cost. The higher levels are expensive, but fast. As we move down the hierarchy, the cost per bit decreases, whereas the access time increases. This trade-off is reasonable; if a given storage system were both faster and less expensive than another—other properties being the same—then there would be no reason to use the slower, more expensive memory.

The fastest storage media—for example, cache and main memory—are referred to as **primary storage**. The media in the next level in the hierarchy—for example, flash memory and magnetic disks—are referred to as **secondary storage**, or **online storage**. The media in the lowest level in the hierarchy—for example, magnetic tape and optical-disk jukeboxes—are referred to as **tertiary storage**, or **offline storage**.

In addition to the speed and cost of the various storage systems, there is also the issue of storage volatility. In the hierarchy shown in Figure 12.1, the storage systems

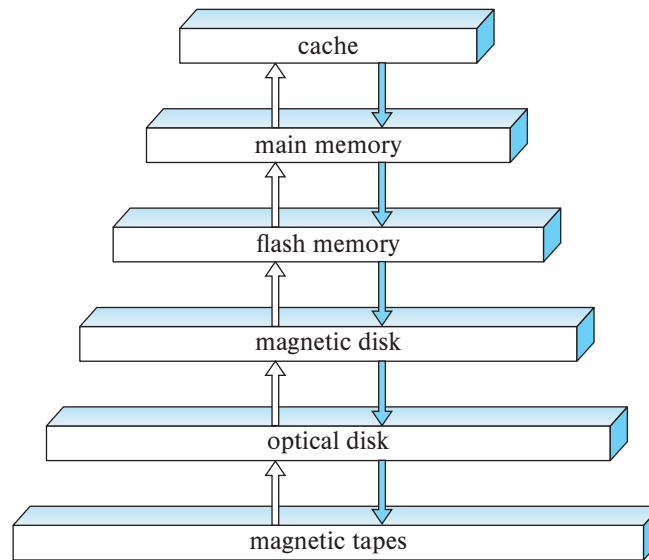


Figure 12.1 Storage device hierarchy.

from main memory up are volatile, whereas the storage systems from flash memory down are non-volatile. Data must be written to non-volatile storage for safekeeping. We shall return to the subject of safe storage of data in the face of system failures later, in Chapter 19.

12.2 Storage Interfaces

Magnetic disks as well as flash-based solid-state disks are connected to a computer system through a high-speed interconnection. Disks typically support either the **Serial ATA (SATA)** interface, or the **Serial Attached SCSI (SAS)** interface; the SAS interface is typically used only in servers. The SATA-3 version of SATA nominally supports 6 gigabytes per second, allowing data transfer speeds of up to 600 megabytes per second, while SAS version 3 supports data transfer rates of 12 gigabits per second. The **Non-Volatile Memory Express (NVMe)** interface is a logical interface standard developed to better support SSDs and is typically used with the PCIe interface (the PCIe interface provides high-speed data transfer internal to computer systems).

While disks are usually connected directly by cables to the disk interface of the computer system, they can be situated remotely and connected by a high-speed network to the computer. In the **storage area network (SAN)** architecture, large numbers of disks are connected by a high-speed network to a number of server computers. The disks are usually organized locally using a storage organization technique called *redundant arrays of independent disks* (RAID) (described later, in Section 12.5), to give the servers a logical view of a very large and very reliable disk. Interconnection technologies used

in storage area networks include iSCSI, which allows SCSI commands to be sent over an IP network, Fiber Channel FC, which supports transfer rates of 1.6 to 12 gigabytes per second, depending on the version, and InfiniBand, which provides very low latency high-bandwidth network communication.

Network attached storage (NAS) is an alternative to SAN. NAS is much like SAN, except that instead of the networked storage appearing to be a large disk, it provides a file system interface using networked file system protocols such as NFS or CIFS. Recent years have also seen the growth of **cloud storage**, where data are stored in the cloud and accessed via an API. Cloud storage has a very high latency of tens to hundreds of milliseconds, if the data are not co-located with the database, and is thus not ideal as the underlying storage for databases. However, applications often use cloud storage for storing objects. Cloud-based storage systems are discussed further in Section 21.7.

12.3 Magnetic Disks

Magnetic disks provide the bulk of secondary storage for modern computer systems. Magnetic disk capacities have been growing steadily year after year, but the storage requirements of large applications have also been growing very fast, in some cases even faster than the growth rate of disk capacities. Very large databases at “web-scale” require thousands to tens of thousands of disks to store their data.¹

In recent years, SSD storage sizes have grown rapidly, and the cost of SSDs has come down significantly; the increasing affordability of SSDs coupled with their much better performance has resulted in SSDs increasingly becoming a competitor to magnetic disk storage for several applications. However, the fact that the per-byte cost of storage on SSDs is around six to eight times the per-byte cost of storage on magnetic disks means that magnetic disks continue to be the preferred choice for storing very large volumes of data in many applications. Example of such data include video and image data, as well as data that is accessed less frequently, such as user-generated data in many web-scale applications. SSDs have however, increasingly become the preferred choice for enterprise data.

12.3.1 Physical Characteristics of Disks

Figure 12.2 shows a schematic diagram of a magnetic disk, while Figure 12.3 shows the internals of an actual magnetic disk. Each disk **platter** has a flat, circular shape. Its two surfaces are covered with a magnetic material, and information is recorded on the surfaces. Platters are made from rigid metal or glass.

When the disk is in use, a drive motor spins it at a constant high speed, typically 5400 to 10,000 revolutions per minute, depending on the model. There is a read-write head positioned just above the surface of the platter. The disk surface is logically di-

¹ We study later, in Chapter 21, how to partition such large amounts of data across multiple nodes in a parallel computing system.

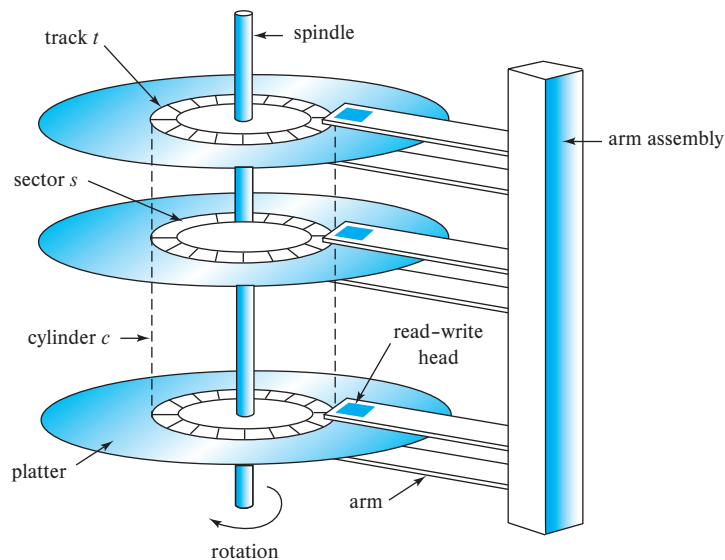


Figure 12.2 Schematic diagram of a magnetic disk.

vided into **tracks**, which are subdivided into **sectors**. A **sector** is the smallest unit of information that can be read from or written to the disk. Sector sizes are typically 512 bytes, and current generation disks have between 2 billion and 24 billion sectors. The inner tracks (closer to the spindle) are of smaller length than the outer tracks, and the outer tracks contain more sectors than the inner tracks.

The **read-write head** stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material.

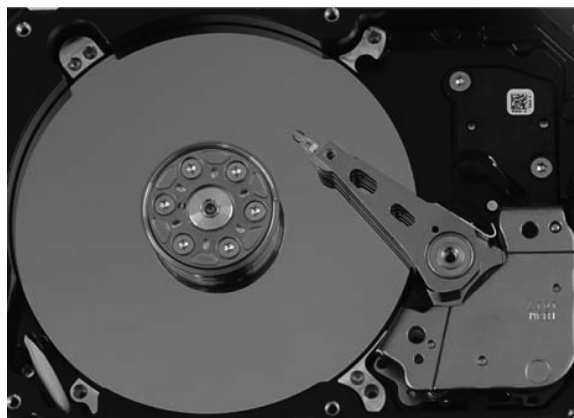


Figure 12.3 Internals of an actual magnetic disk.

Each side of a platter of a disk has a read-write head that moves across the platter to access different tracks. A disk typically contains many platters, and the read-write heads of all the tracks are mounted on a single assembly called a **disk arm** and move together. The disk platters mounted on a spindle and the heads mounted on a disk arm are together known as **head-disk assemblies**. Since the heads on all the platters move together, when the head on one platter is on the i th track, the heads on all other platters are also on the i th track of their respective platters. Hence, the i th tracks of all the platters together are called the i th **cylinder**.

The read-write heads are kept as close as possible to the disk surface to increase the recording density. The head typically floats or flies only microns from the disk surface; the spinning of the disk creates a small breeze, and the head assembly is shaped so that the breeze keeps the head floating just above the disk surface. Because the head floats so close to the surface, platters must be machined carefully to be flat.

Head crashes can be a problem. If the head contacts the disk surface, the head can scrape the recording medium off the disk, destroying the data that had been there. In older-generation disks, the head touching the surface caused the removed medium to become airborne and to come between the other heads and their platters, causing more crashes; a head crash could thus result in failure of the entire disk. Current-generation disk drives use a thin film of magnetic metal as recording medium. They are much less susceptible to failure of the entire disk, but are susceptible to failure of individual sectors.

A **disk controller** interfaces between the computer system and the actual hardware of the disk drive; in modern disk systems, the disk controller is implemented within the disk drive unit. A disk controller accepts high-level commands to read or write a sector, and initiates actions, such as moving the disk arm to the right track and actually reading or writing the data. Disk controllers also attach **checksums** to each sector that is written; the checksum is computed from the data written to the sector. When the sector is read back, the controller computes the checksum again from the retrieved data and compares it with the stored checksum; if the data are corrupted, with a high probability the newly computed checksum will not match the stored checksum. If such an error occurs, the controller will retry the read several times; if the error continues to occur, the controller will signal a read failure.

Another interesting task that disk controllers perform is **remapping of bad sectors**. If the controller detects that a sector is damaged when the disk is initially formatted, or when an attempt is made to write the sector, it can logically map the sector to a different physical location (allocated from a pool of extra sectors set aside for this purpose). The remapping is noted on disk or in non-volatile memory, and the write is carried out on the new location.

12.3.2 Performance Measures of Disks

The main measures of the qualities of a disk are capacity, access time, data-transfer rate, and reliability.

Access time is the time from when a read or write request is issued to when data transfer begins. To access (i.e., to read or write) data on a given sector of a disk, the arm first must move so that it is positioned over the correct track, and then must wait for the sector to appear under it as the disk rotates. The time for repositioning the arm is called the **seek time**, and it increases with the distance that the arm must move. Typical seek times range from 2 to 20 milliseconds depending on how far the track is from the initial arm position. Smaller disks tend to have lower seek times since the head has to travel a smaller distance.

The **average seek time** is the average of the seek times, measured over a sequence of (uniformly distributed) random requests. If all tracks have the same number of sectors, and we disregard the time required for the head to start moving and to stop moving, we can show that the average seek time is one-third the worst-case seek time. Taking these factors into account, the average seek time is around one-half of the maximum seek time. Average seek times currently range between 4 and 10 milliseconds, depending on the disk model.²

Once the head has reached the desired track, the time spent waiting for the sector to be accessed to appear under the head is called the **rotational latency time**. Rotational speeds of disks today range from 5400 rotations per minute (90 rotations per second) up to 15,000 rotations per minute (250 rotations per second), or, equivalently, 4 milliseconds to 11.1 milliseconds per rotation. On an average, one-half of a rotation of the disk is required for the beginning of the desired sector to appear under the head. Thus, the **average latency time** of the disk is one-half the time for a full rotation of the disk. Disks with higher rotational speeds are used for applications where latency needs to be minimized.

The access time is then the sum of the seek time and the latency; average access times range from 5 to 20 milliseconds depending on the disk model. Once the first sector of the data to be accessed has come under the head, data transfer begins. The **data-transfer rate** is the rate at which data can be retrieved from or stored to the disk. Current disk systems support maximum transfer rates of 50 to 200 megabytes per second; transfer rates are significantly lower than the maximum transfer rates for inner tracks of the disk, since they have fewer sectors. For example, a disk with a maximum transfer rate of 100 megabytes per second may have a sustained transfer rate of around 30 megabytes per second on its inner tracks.

Requests for disk I/O are typically generated by the file system but can be generated directly by the database system. Each request specifies the address on the disk to be referenced; that address is in the form of a *block number*. A **disk block** is a logical unit of storage allocation and retrieval, and block sizes today typically range from 4 to 16

²Smaller 2.5-inch diameter disks have a lesser arm movement distance than larger 3.5-inch disks, and thus have lower seek times. As a result 2.5-inch disks have been the preferred choice for applications where latency needs to be minimized, although SSDs are increasingly preferred for such applications. Larger 3.5-inch diameter disks have a lower cost per byte and are used in data storage applications where cost is an important factor.

kilobytes. Data are transferred between disk and main memory in units of blocks. The term **page** is often used to refer to blocks, although in a few contexts (such as flash memory) they refer to different things.

A sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. In a **sequential access** pattern, successive requests are for successive block numbers, which are on the same track, or on adjacent tracks. To read blocks in sequential access, a disk seek may be required for the first block, but successive requests would either not require a seek, or require a seek to an adjacent track, which is faster than a seek to a track that is farther away. Data transfer rates are highest with a sequential access pattern, since seek time is minimal.

In contrast, in a **random access** pattern, successive requests are for blocks that are randomly located on disk. Each such request would require a seek. The number of **I/O operations per second (IOPS)**, that is, the number random block accesses that can be satisfied by a disk in a second, depends on the access time, and the block size, and the data transfer rate of the disk. With a 4-kilobyte block size, current generation disks support between 50 and 200 IOPS, depending on the model. Since only a small amount (one block) of data are read per seek, the data transfer rate is significantly lower with a random access pattern than with a sequential access pattern.

The final commonly used measure of a disk is the **mean time to failure (MTTF)**,³ which is a measure of the reliability of the disk. The mean time to failure of a disk (or of any other system) is the amount of time that, on average, we can expect the system to run continuously without any failure. According to vendors' claims, the mean time to failure of disks today ranges from 500,000 to 1,200,000 hours—about 57 to 136 years. In practice the claimed mean time to failure is computed on the probability of failure when the disk is new—the figure means that given 1000 relatively new disks, if the MTTF is 1,200,000 hours, on an average one of them will fail in 1200 hours. A mean time to failure of 1,200,000 hours does not imply that the disk can be expected to function for 136 years! Most disks have an expected life span of about 5 years and have significantly higher rates of failure once they become more than a few years old.

12.4 Flash Memory

There are two types of flash memory, NOR flash and NAND flash. NAND flash is the variant that is predominantly used for data storage. Reading from NAND flash requires an entire *page* of data, which is very commonly 4096 bytes, to be fetched from NAND flash into main memory. Pages in a NAND flash are thus similar to sectors in a magnetic disk.

³The term **mean time between failures (MTBF)** is often used to refer to MTTF in the context of disk drives, although technically MTBF should only be used in the context of systems that can be repaired after failure, and may fail again; MTBF would then be the sum of MTTF and the mean time to repair. Magnetic disks can almost never be repaired after a failure.

Solid-state disks (SSDs) are built using NAND flash and provide the same block-oriented interface as disk storage. Compared to magnetic disks, SSDs can provide much faster random access: the latency to retrieve a page of data ranges from 20 to 100 microseconds for SSDs, whereas a random access on disk would take 5 to 10 milliseconds. The data transfer rate of SSDs is higher than that of magnetic disks and is usually limited by the interconnect technology; transfer rates range from around 500 megabytes per second with SATA interfaces, up to 3 gigabytes per second using NVMe PCIe interfaces, depending on the specific SSD model, in contrast to a maximum of about 200 megabytes per second with magnetic disk. The power consumption of SSDs is also significantly lower than that of magnetic disks.

Writes to flash memory are a little more complicated. A write to a page of flash memory typically takes about 100 microseconds. However, once written, a page of flash memory cannot be directly overwritten. Instead, it has to be erased and rewritten subsequently. The erase operation must be performed on a group of pages, called an **erase block**, erasing all the pages in the block, and takes about 2 to 5 milliseconds. An erase block (often referred to as just “block” in flash literature), is typically 256 kilobytes to 1 megabyte, and contains around 128 to 256 pages. Further, there is a limit to how many times a flash page can be erased, typically around 100,000 to 1,000,000 times. Once this limit is reached, errors in storing bits are likely to occur.

Flash memory systems limit the impact of both the slow erase speed and the update limits by mapping logical page numbers to physical page numbers. When a logical page is updated, it can be remapped to any already erased physical page, and the original location can be erased later. Each physical page has a small area of memory where its logical address is stored; if the logical address is remapped to a different physical page, the original physical page is marked as deleted. Thus, by scanning the physical pages, we can find where each logical page resides. The logical-to-physical page mapping is replicated in an in-memory **translation table** for quick access.

Blocks containing multiple deleted pages are periodically erased, taking care to first copy nondeleted pages in those blocks to a different block (the translation table is updated for these nondeleted pages). Since each physical page can be updated only a fixed number of times, physical pages that have been erased many times are assigned “cold data,” that is, data that are rarely updated, while pages that have not been erased many times are used to store “hot data,” that is, data that are updated frequently. This principle of evenly distributing erase operations across physical blocks is called **wear leveling** and is usually performed transparently by flash-memory controllers. If a physical page is damaged due to an excessive number of updates, it can be removed from usage, without affecting the flash memory as a whole.

All the above actions are carried out by a layer of software called the **flash translation layer**; above this layer, flash storage looks identical to magnetic disk storage, providing the same page/sector-oriented interface, except that flash storage is much faster. File systems and database storage structures can thus see an identical logical view of the underlying storage structure, regardless of whether it is flash or magnetic storage.

Note 12.1 STORAGE CLASS MEMORY

Although flash is the most widely used type of non-volatile memory, there have been a number of alternative non-volatile memory technologies developed over the years. Several of these technologies allow direct read and write access to individual bytes or words, avoiding the need to read or write in units of pages (and also avoiding the erase overhead of NAND flash). Such types of non-volatile memory are referred to as **storage class memory**, since they can be treated as a large non-volatile block of memory. The 3D-XPoint memory technology, developed by Intel and Micron, is a recently developed storage class memory technology. In terms of cost per byte, latency of access, and capacity, 3D-XPoint memory lies in between main memory and flash memory. Intel Optane SSDs based on 3D-XPoint started shipping in 2017, and Optane persistent memory modules were announced in 2018.

SSD performance is usually expressed in terms of:

1. The *number of random block reads per second*, with 4-kilobyte blocks being the standard. Typical values in 2018 are about 10,000 random reads per second (also referred to as 10,000 IOPS) with 4-kilobyte blocks, although some models support higher rates.

Unlike magnetic disks, SSDs can support multiple random requests in parallel, with 32 parallel requests being commonly supported; a flash disk with SATA interface supports nearly 100,000 random 4-kilobyte block reads in a second with 32 requests sent in parallel, while SSDs connected using NVMe PCIe can support over 350,000 random 4-kilobyte block reads per second. These numbers are specified as QD-1 for rates without parallelism and QD-n for n-way parallelism, with QD-32 being the most commonly used number.

2. The *data transfer rate* for sequential reads and sequential writes. Typical rates for both sequential reads and sequential writes are 400 to 500 megabytes per second for SSDs with a SATA 3 interface, and 2 to 3 gigabytes per second for SSDs using NVMe over the PCIe 3.0x4 interface.
3. The *number of random block writes per second*, with 4-kilobyte blocks being the standard. Typical values in 2018 are about 40,000 random 4-kilobyte writes per second for QD-1 (without parallelism), and around 100,000 IOPS for QD-32, although some models support higher rates for both QD-1 and QD-32.

Hybrid disk drives are hard-disk systems that combine magnetic storage with a smaller amount of flash memory, which is used as a cache for frequently accessed data. Frequently accessed data that are rarely updated are ideal for caching in flash memory.

An approach to storing data based on storing all values of a particular column together, rather than storing all attributes of a particular row together, has been found to work very well for analytical query processing. This idea, called *column-oriented storage*, is discussed in Section 13.6.

Some applications need very fast access to data and have small enough data sizes that the entire database can fit into the main memory of a database server machine. In such cases, we can keep a copy of the entire database in memory.¹ Databases that store the entire database in memory and optimize in-memory data structures as well as query processing and other algorithms used by the database to exploit the memory residency of data are called **main-memory databases**. Storage organization in main-memory databases is discussed in Section 13.7. We note that non-volatile memory that allows direct access to individual bytes or cache lines, called *storage class memory*, is under development. Main-memory database architectures can be further optimized for such storage.

13.2 File Organization

A database is mapped into a number of different files that are maintained by the underlying operating system. These files reside permanently on disks. A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called **blocks**, which are the units of both storage allocation and data transfer. Most databases use block sizes of 4 to 8 kilobytes by default, but many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications.

A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that *no record is larger than a block*. This assumption is realistic for most data-processing applications, such as our university example. There are certainly several kinds of large data items, such as images, that can be significantly larger than a block. We briefly discuss how to handle such large data items in Section 13.2.2, by storing large data items separately, and storing a pointer to the data item in the record.

In addition, we shall require that *each record is entirely contained in a single block*; that is, no record is contained partly in one block, and partly in another. This restriction simplifies and speeds up access to data items.

¹To be safe, not only should the current database fit in memory, but there should be a reasonable certainty that the database will continue to fit in memory in the medium term future, despite potential growth of the organization.

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records and consider storage of variable-length records later.

13.2.1 Fixed-Length Records

As an example, let us consider a file of *instructor* records for our university database. Each record of this file is defined (in pseudocode) as:

```

type instructor = record
    ID varchar (5);
    name varchar(20);
    dept_name varchar (20);
    salary numeric (8,2);
end

```

Assume that each character occupies 1 byte and that numeric (8,2) occupies 8 bytes. Suppose that instead of allocating a variable amount of bytes for the attributes *ID*, *name*, and *dept_name*, we allocate the maximum number of bytes that each attribute can hold. Then, the *instructor* record is 53 bytes long. A simple approach is to use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on (Figure 13.1).

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.1 File containing *instructor* records.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.2 File of Figure 13.1, with record 3 deleted and all records moved.

However, there are two problems with this simple approach:

1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block (this number can be computed easily by dividing the block size by the record size, and discarding the fractional part). Any remaining bytes of each block are left unused.

When a record is deleted, we could move the record that comes after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 13.2). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 13.3).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure 13.3 File of Figure 13.1, with record 3 deleted and final record moved.

to store there is the address of the first record whose contents are deleted. We use this first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 13.4 shows the file of Figure 13.1, with the free list, after records 1, 4, and 6 have been deleted.

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file.

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.4 File of Figure 13.1, with free list after deletion of records 1, 4, and 6.

Insertion and deletion for files of fixed-length records are simple to implement because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable-length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

13.2.2 Variable-Length Records

Variable-length records arise in database systems due to several reasons. The most common reason is the presence of variable length fields, such as strings. Other reasons include record types that contain repeating fields such as arrays or multisets, and the presence of multiple record types within a file.

Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

- 1. How to represent a single record in such a way that individual attributes can be extracted easily, even if they are of variable length
- 2. How to store variable-length records within a block, such that records in a block can be extracted easily

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length information, whose structure is the same for all records of the same relation, followed by the contents of variable-length attributes. Fixed-length attributes, such as numeric values, dates, or fixed-length character strings are allocated as many bytes as required to store their value. Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (*offset*, *length*), where *offset* denotes where the data for that attribute begins within the record, and *length* is the length in bytes of the variable-sized attribute. The values for the variable-length attributes are stored consecutively, after the initial fixed-length part of the record. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length.

An example of such a record representation is shown in Figure 13.5. The figure shows an *instructor* record whose first three attributes *ID*, *name*, and *dept_name* are variable-length strings, and whose fourth attribute *salary* is a fixed-sized number. We assume that the offset and length values are stored in two bytes each, for a total of 4

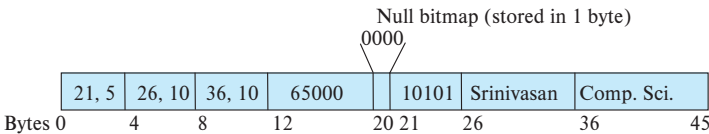


Figure 13.5 Representation of a variable-length record of the *instructor* relation.

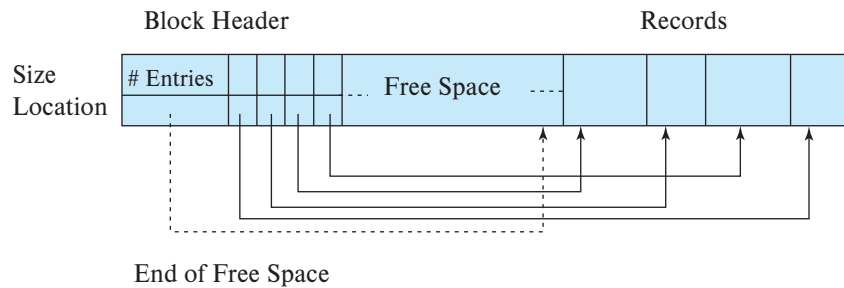


Figure 13.6 Slotted-page structure.

bytes per attribute. The *salary* attribute is assumed to be stored in 8 bytes, and each string takes as many bytes as it has characters.

The figure also illustrates the use of a **null bitmap**, which indicates which attributes of the record have a null value. In this particular record, if the *salary* were null, the fourth bit of the bitmap would be set to 1, and the *salary* value stored in bytes 12 through 19 would be ignored. Since the record has four attributes, the null bitmap for this record fits in 1 byte, although more bytes may be required with more attributes. In some representations, the null bitmap is stored at the beginning of the record, and for attributes that are null, no data (value, or offset/length) are stored at all. Such a representation would save some storage space, at the cost of extra work to extract attributes of the record. This representation is particularly useful for certain applications where records have a large number of fields, most of which are null.

We next address the problem of storing variable-length records in a block. The **slotted-page structure** is commonly used for organizing records within a block and is shown in Figure 13.6.² There is a header at the beginning of each block, containing the following information:

- The number of record entries in the header
- The end of free space in the block
- An array whose entries contain the location and size of each record

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous between the final entry in the header array and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to *deleted* (its size is set to -1 , for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all

²Here, “page” is synonymous with “block.”

free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: typical values are around 4 to 8 kilobytes.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

13.2.3 Storing Large Objects

Databases often store data that can be much larger than a disk block. For instance, an image or an audio recording may be multiple megabytes in size, while a video object may be multiple gigabytes in size. Recall that SQL supports the types **blob** and **clob**, which store binary and character large objects.

Many databases internally restrict the size of a record to be no larger than the size of a block.³ These databases allow records to logically contain large objects, but they store the large objects separate from the other (short) attributes of records in which they occur. A (logical) pointer to the object is then stored in the record containing the large object.

Large objects may be stored either as files in a file system area managed by the database, or as file structures stored in and managed by the database. In the latter case, such in-database large objects can optionally be represented using B⁺-tree file organizations, which we study in Section 14.4.1, to allow efficient access to any location within the object. B⁺-tree file organizations permit us to read an entire object, or specified byte ranges in the object, as well as to insert and delete parts of the object.

However, there are some performance issues with storing very large objects in databases. The efficiency of accessing large objects via database interfaces is one concern. A second concern is the size of database backups. Many enterprises periodically create “database dumps,” that is, backup copies of their databases; storing large objects in the database can result in a large increase in the size of the database dumps.

Many applications therefore choose to store very large objects, such as video data, outside of the database, in a file system. In such cases, the application may store the file name (usually a path in the file system) as an attribute of a record in the database. Storing data in files outside the database can result in file names in the database pointing to files that do not exist, perhaps because they have been deleted, which results in a form of foreign-key constraint violation. Further, database authorization controls are not applicable to data stored in the file system.

³This restriction helps simplify buffer management; as we see in Section 13.5, disk blocks are brought into an area of memory called the buffer before they are accessed. Records larger than a block would get split between blocks, which may be different areas of the buffer, and thus cannot be guaranteed to be in a contiguous area of memory.

Some databases support file system integration with the database, to ensure that constraints are satisfied (for example, deletion of files will be blocked if the database has a pointer to the file), and to ensure that access authorizations are enforced. Files can be accessed both from a file system interface and from the database SQL interface. For example, Oracle supports such integration through its SecureFiles and Database File System features.

13.3 Organization of Records in Files

So far, we have studied how records are represented in a file structure. A relation is a set of records. Given a set of records, the next question is how to organize them in a file. Several of the possible ways of organizing records in files are:

- **Heap file organization.** Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is either a single file or a set of files for each relation. Heap file organization is discussed in Section 13.3.1.
- **Sequential file organization.** Records are stored in sequential order, according to the value of a “search key” of each record. Section 13.3.2 describes this organization.
- **Multitable clustering file organization:** Generally, a separate file or set of files is used to store the records of each relation. However, in a *multitable clustering file organization*, records of several different relations are stored in the same file, and in fact in the same block within a file, to reduce the cost of certain join operations. Section 13.3.3 describes the multitable clustering file organization.
- **B⁺-tree file organization.** The traditional sequential file organization described in Section 13.3.2 does support ordered access even if there are insert, delete, and update operations, which may change the ordering of records. However, in the face of a large number of such operations, efficiency of ordered access suffers. We study another way of organizing records, called the *B⁺-tree file organization*, in Section 14.4.1. The B⁺-tree file organization is related to the B⁺-tree index structure described in that chapter and can provide efficient ordered access to records even if there are a large number of insert, delete, or update operations. Further, it supports very efficient access to specific records, based on the search key.
- **Hashing file organization.** A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed. Section 14.5 describes this organization; it is closely related to the indexing structures described in that chapter.

13.3.1 Heap File Organization

In a heap file organization, a record may be stored anywhere in the file corresponding to a relation. Once placed in a particular location, the record is not usually moved.⁴

When a record is inserted in a file, one option for choosing the location is to always add it at the end of the file. However, if records get deleted, it makes sense to use the space thus freed up to store new records. It is important for a database system to be able to efficiently find blocks that have free space, without having to sequentially search through all the blocks of the file.

Most databases use a space-efficient data structure called a **free-space map** to track which blocks have free space to store records. The free-space map is commonly represented by an array containing 1 entry for each block in the relation. Each entry represents a fraction f such that at least a fraction f of the space in the block is free. In PostgreSQL, for example, an entry is 1 byte, and the value stored in the entry must be divided by 256 to get the free-space fraction. The array is stored in a file, whose blocks are fetched into memory,⁵ as required. Whenever a record is inserted, deleted, or changed in size, if the occupancy fraction changes enough to affect the entry value, the entry is updated in the free-space map. An example of a free-space map for a file with 16 blocks is shown below. We assume that 3 bits are used to store the occupancy fraction; the value at position i should be divided by 8 to get the free-space fraction for block i .

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For example, a value of 7 indicates that at least $7/8$ th of the space in the block is free.

To find a block to store a new record of a given size, the database can scan the free-space map to find a block that has enough free space to store that record. If there is no such block, a new block is allocated for the relation.

While such a scan is much faster than actually fetching blocks to find free space, it can still be very slow for large files. To further speed up the task of locating a block with sufficient free space, we can create a second-level free-space map, which has, say 1 entry for every 100 entries of the main free-space map. That 1 entry stores the maximum value amongst the 100 entries in the main free-space map that it corresponds to. The free-space map below is a second level free-space map for our earlier example, with 1 entry for every 4 entries in the main free-space map.

4	7	2	6
---	---	---	---

⁴Records may be occasionally moved, for example, if the database sorts the records of the relation; but note that even if the relation is reordered by sorting, subsequent insertions and updates may result in the records no longer being ordered.

⁵Via the database buffer, which we discuss in Section 13.5.

With 1 entry for every 100 entries in the main free-space map, a scan of the second-level free space map would take only 1/100th of the time to scan the main free-space map; once a suitable entry indicating enough free space is found there, its corresponding 100 entries in the main free-space map can be examined to find a block with sufficient free space. Such a block must exist, since the second-level free-space map entry stores the maximum of the entries in the main free-space map. To deal with very large relations, we can create more levels beyond the second level, using the same idea.

Writing the free-space map to disk every time an entry in the map is updated would be very expensive. Instead, the free-space map is written periodically; as a result, the free-space map on disk may be outdated, and when a database starts up, it may get outdated data about available free space. The free-space map may, as a result, claim a block has free space when it does not; such an error will be detected when the block is fetched, and can be dealt with by a further search in the free-space map to find another block. On the other hand, the free-space map may claim that a block does not have free space when it does; generally this will not result in any problem other than unused free space. To fix any such errors, the relation is scanned periodically and the free-space map recomputed and written to disk.

13.3.2 Sequential File Organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Figure 13.7 shows a sequential file of *instructor* records taken from our university example. In that example, the records are stored in search-key order, using *ID* as the search key.

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes, as well as for certain query-processing algorithms that we shall study in Chapter 15.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. We can manage deletion by using pointer chains, as we saw previously. For insertion, we apply the following two rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.
2. If there is a free record (i.e., space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

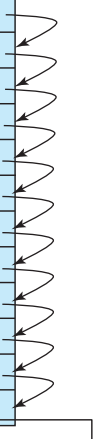


Figure 13.7 Sequential file for *instructor* records.

an *overflow block*. In either case, adjust the pointers so as to chain together the records in search-key order.

Figure 13.8 shows the file of Figure 13.7 after the insertion of the record (32222, Verdi, Music, 48000). The structure in Figure 13.8 allows fast insertion of new records, but it forces sequential file-processing applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. Eventually, however, the correspondence between search-key order and physical order may be totally lost over a period of time, in which case sequential processing will become much less efficient. At this point, the file should be **reorganized** so that it is once again physically in sequential order. Such reorganizations are costly and must be done during times when the system load is low. The frequency with which reorganizations are needed depends on the frequency of insertion of new records. In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field in Figure 13.7 is not needed.

The B⁺-tree file organization, which we describe in Section 14.4.1, provides efficient ordered access even if there are many inserts, deletes, and updates, without requiring expensive reorganizations.

13.3.3 Multitable Clustering File Organization

Most relational database systems store each relation in a separate file, or a separate set of files. Thus, each file, and as a result, each block, stores records of only one relation, in such a design.

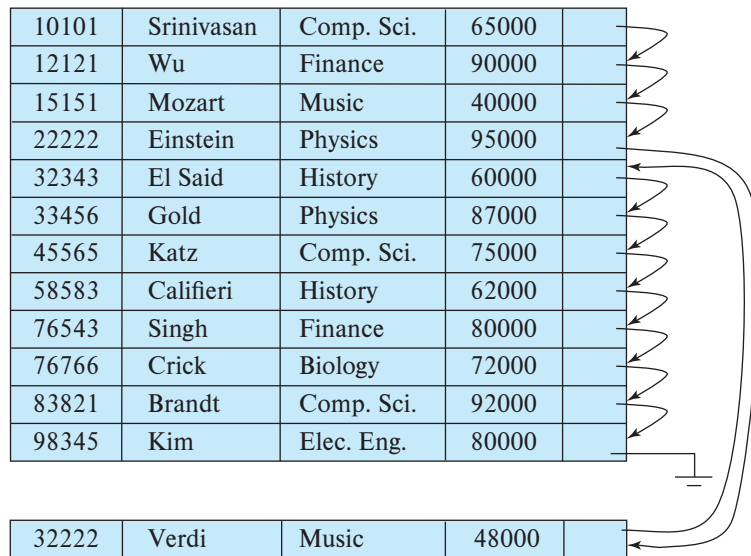


Figure 13.8 Sequential file after an insertion.

However, in some cases it can be useful to store records of more than one relation in a single block. To see the advantage of storing records of multiple relations in one block, consider the following SQL query for the university database:

```
select dept_name, building, budget, ID, name, salary
from department natural join instructor;
```

This query computes a join of the *department* and *instructor* relations. Thus, for each tuple of *department*, the system must locate the *instructor* tuples with the same value for *dept_name*. Ideally, these records will be located with the help of *indices*, which we shall discuss in Chapter 14. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

As a concrete example, consider the *department* and *instructor* relations of Figure 13.9 and Figure 13.10, respectively (for brevity, we include only a subset of the tuples

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

Figure 13.9 The *department* relation.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Figure 13.10 The *instructor* relation.

of the relations we have used thus far). In Figure 13.11, we show a file structure designed for the efficient execution of queries involving the natural join of *department* and *instructor*. All the *instructor* tuples for a particular *dept_name* are stored near the *department* tuple for that *dept_name*. We say that the two relations are clustered on the key *dept_name*. We assume that each record contains the identifier of the relation to which it belongs, although this is not shown in Figure 13.11.

Although not depicted in the figure, it is possible to store the value of the *dept_name* attribute, which defines the clustering, only once for a group of tuples (from both relations), reducing storage overhead.

This structure allows for efficient processing of the join. When a tuple of the *department* relation is read, the entire block containing that tuple is copied from disk into main memory. Since the corresponding *instructor* tuples are stored on the disk near the *department* tuple, the block containing the *department* tuple contains tuples of the *instructor* relation needed to process the query. If a department has so many instructors that the *instructor* records do not fit in one block, the remaining records appear on nearby blocks.

A **multitable clustering file organization** is a file organization, such as that illustrated in Figure 13.11, that stores related records of two or more relations in each block.⁶ The **cluster key** is the attribute that defines which records are stored together; in our preceding example, the cluster key is *dept_name*.

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Figure 13.11 Multitable clustering file structure.

⁶Note that the word *cluster* is often used to refer to a group of machines that together constitute a parallel database; that use of the word *cluster* is unrelated to the concept of multitable clustering.

Although a multitable clustering file organization can speed up certain join queries, it can result in slowing processing of other types of queries. For example, in our preceding example,

```
select *
from department;
```

requires more block accesses than it did in the scheme under which we stored each relation in a separate file, since each block now contains significantly fewer *department* records. To locate efficiently all tuples of the *department* relation within a particular block, we can chain together all the records of that relation using pointers; however, the number of blocks read does not get affected by using such chains.

When multitable clustering is to be used depends on the types of queries that the database designer believes to be most frequent. Careful use of multitable clustering can produce significant performance gains in query processing.

Multitable clustering is supported by the Oracle database system. Clusters can be created by using a **create cluster** command, with a specified cluster key. An extension of the **create table** command can be used to specify that a relation is to be stored in a specific cluster, with a particular attribute used as the cluster key. Multiple relations can thus be allocated to a cluster.

13.3.4 Partitioning

Many databases allow the records in a relation to be partitioned into smaller relations that are stored separately. Such **table partitioning** is typically done on the basis of an attribute value; for example, records in a *transaction* relation in an accounting database may be partitioned by year into smaller relations corresponding to each year, such as *transaction_2018*, *transaction_2019*, and so on. Queries can be written based on the *transaction* relation but are translated into queries on the year-wise relations. Most accesses are to records of the current year and include a selection based on the year. Query optimizers can rewrite such a query to only access the smaller relation corresponding to the requested year, and they can avoid reading records corresponding to other years. For example, a query

```
select *
from transaction
where year=2019
```

would only access the relation *transaction_2019*, ignoring the other relations, while a query without the selection condition would read all the relations.

The cost of some operations, such as finding free space for a record, increase with relation size; by reducing the size of each relation, partitioning helps reduce such overheads. Partitioning can also be used to store different parts of a relation on different